LocalFix

End-of-Year Project

Amine **Khiareddine**
Mohamed Aziz **Hamida**
Mohamed Omar **Taamalli**

April 19, 2026

Contents

General Introduction **4**

1 General Context and Preliminary Study **5**
1.1 Introduction . . . 5
1.2 General Framework . . . 5
1.3 Host Organization . . . 5
1.4 Project Presentation . . . 5
1.4.1 The Problem . . . 5
1.4.2 A Look at Existing Platforms . . . 6
1.4.3 Comparing the Two Platforms . . . 6
1.4.4 Our Proposed Solution . . . 7
1.5 Project Management Methodology . . . 7
1.5.1 The SCRUM Framework . . . 7
1.5.2 Roles in SCRUM . . . 8
1.5.3 SCRUM Artifacts . . . 8
1.5.4 SCRUM Events . . . 9
1.6 Conclusion . . . 9

2 Analysis of Needs and Specification **10**
2.1 Introduction . . . 10
2.2 Project Sprints . . . 10
2.3 Analysis of Needs and Specification . . . 10
2.3.1 Identifying the Actors . . . 10
2.3.2 Functional Requirements . . . 10
2.3.3 Non-Functional Requirements . . . 12
2.3.4 Work Planning with SCRUM . . . 12
2.4 Composition of the Product Backlog . . . 12
2.4.1 Sprints Breakdown . . . 13
2.5 Application Architecture . . . 13
2.5.1 Logical Architecture . . . 13
2.5.2 Physical Architecture . . . 15
2.6 Software Environment of the Project . . . 15
2.6.1 Project Management Tools . . . 15
2.6.2 Development and Testing Tools . . . 15
2.7 Conclusion . . . 16

3 Release 1 **17**
3.1 Introduction . . . 17
3.2 Sprint 1: Task and User Management . . . 17
3.2.1 Sprint 1 Objective . . . 17
3.2.2 Analysis of Sprint 1 . . . 17
3.2.3 Design of Sprint 1 . . . 19
3.2.4 Completion of Sprint 1 . . . 20
3.3 Sprint 2: User Management . . . 23
3.3.1 Objective of Sprint 2 . . . 23
3.3.2 Backlog of Sprint 2 . . . 23
3.3.3 Analysis of Sprint 2 . . . 23
3.3.4 Design of Sprint 2 . . . 25
3.3.5 Completion of Sprint 2 . . . 26
3.4 Conclusion . . . 27

4 Release 2 **28**
4.1 Introduction . . . 28
4.2 Sprint 3: Interaction, Task Status, and Platform Visibility . . . 28
4.2.1 Objective of Sprint 3 . . . 28
4.2.2 Backlog of Sprint 3 . . . 28
4.2.3 Analysis of Sprint 3 . . . 28
4.2.4 Design of Sprint 3 . . . 30
4.2.5 Completion of Sprint 3 . . . 31
4.3 Sprint 4: Rating & Review System and General Polish . . . 33
4.3.1 Objective of Sprint 4 . . . 33
4.3.2 Backlog of Sprint 4 . . . 34
4.3.3 Analysis of Sprint 4 . . . 34
4.3.4 Design of Sprint 4 . . . 35
4.3.5 Completion of Sprint 4 . . . 36

General Conclusion and Perspectives **38**

Online Resources **39**

List of Figures

1.1 FSM Logo . . . 5
1.2 Nextdoor Logo . . . 6
1.3 PickMyTask Logo . . . 6
1.4 SCRUM Process . . . 8

2.1 Global use case diagram . . . 11
2.2 MVC Architecture . . . 14
2.3 Event-Driven Architecture . . . 14
2.4 3-Tier Web Architecture . . . 15
2.5 Microsoft Teams Logo . . . 15
2.6 Jira Logo . . . 15
2.7 Node.js Logo . . . 15
2.8 ReactJS Logo . . . 16
2.9 VS Code Logo . . . 16
2.10 MySQL Logo . . . 16
2.11 MongoDB Logo . . . 16
2.12 TeXstudio Logo . . . 16
2.13 Draw.io Logo . . . 16

3.1 Sprint 1 Use Case Diagram . . . 18
3.2 Sprint 1 Class Diagram . . . 19
3.3 Sprint 1 Create Task Sequence Diagram . . . 20
3.4 Authentication Interface . . . 21
3.5 Register Interface . . . 21
3.6 Post-Task Interface . . . 22
3.7 Task Edit/Delete Interface . . . 22
3.8 Task View Interface . . . 23
3.9 Sprint 2 Use Case Diagram . . . 24
3.10 Sprint 2 Class Diagram . . . 25
3.11 Sprint 2 Report Sequence Diagram . . . 26
3.12 Messaging Interface . . . 26
3.13 Reporting Interface . . . 27
3.14 Profile Management Interface . . . 27

4.1 Sprint 3 Use Case Diagram . . . 29
4.2 Sprint 3 Class Diagram . . . 30
4.3 Sprint 3 Sequence Diagram . . . 31
4.4 Notification Interface . . . 31
4.5 Task Board Interface . . . 32
4.6 Atlas (Map View) . . . 32
4.7 Task Register . . . 33
4.8 Accounts Table . . . 33
4.9 Sprint 4 Use Case Diagram . . . 35
4.10 Sprint 4 Class Diagram . . . 36
4.11 Sprint 4 Sequence Diagram . . . 36
4.12 Rating Interface . . . 37
4.13 Review Interface . . . 37

General Introduction

Over the past few years, digital platforms have completely changed the way people connect, work, and get things done. But most of these platforms are built for a global audience — they're big, impersonal, and honestly not that great when you just need quick help with something small and local. That's exactly the gap that **LocalFix** is trying to fill. It's a simple web platform built to connect people who need a hand with others right in their own community who are happy to help.

The idea behind LocalFix is pretty simple: the people around you are often your best resource. Instead of scrolling through huge platforms or random social media groups hoping someone will respond, LocalFix gives you a clean, organized way to ask for help. You can post small tasks — things like getting help with homework, fixing a tech issue, moving some furniture, or running an errand — and quickly find someone nearby who can pitch in. These helpers might be neighbors, classmates, or coworkers; basically anyone close by who has the right skills and a bit of free time.

The platform is designed to be as simple and easy to use as possible. Users can browse a task board, filter tasks by category (like homework, tech support, or errands), and see at a glance whether a task is open, in progress, or done. Posting a task takes just a few clicks, and there's no complicated process to go through.

At the end of the day, LocalFix is about more than just getting tasks done. It's about bringing people a little closer together and making everyday help easier to find. By turning small favors into clear, reliable exchanges, it helps build stronger local connections and a more supportive community overall.

Chapter 1

General Context and Preliminary Study

1.1 Introduction

This chapter lays the groundwork for the whole project. We start with a quick look at the host organization, then get into the details of what LocalFix is actually about. We describe the problem we're trying to solve and what we're hoping to achieve, and we take a look at two existing platforms — Nextdoor and PickMyTask — to understand what they do well and where they fall short.

By the end of the chapter, we present our proposed solution and explain the project management approach we decided to follow, with a focus on the SCRUM framework and how we applied it.

1.2 General Framework

LocalFix was developed as a final-year project for the Higher Technician Diploma in Software Engineering, for the academic year 2025–2026.

1.3 Host Organization

The Faculty of Sciences of Monastir (FSM) is a public university in Tunisia. Its main goal is to provide quality education, help students build strong scientific and technical skills, and support research across different fields. FSM offers several programs, particularly in computer science, mathematics, and physics, combining theory with hands-on practical work.

For this project, FSM served as our host institution, giving us a supportive environment that encourages learning, innovation, and teamwork.



Figure 1.1. FSM Logo

1.4 Project Presentation

1.4.1 The Problem

Existing platforms for local help and task sharing have some clear limitations. Most of them were designed as large marketplaces or social networks, which means that useful help requests end up buried under ads, random posts, and unrelated content. Finding or offering simple, everyday help quickly becomes a chore.

Another issue is that many of these platforms are built around paid services and take commissions, which adds unnecessary complexity and cost. What was supposed to feel like a friendly exchange between neighbors ends up feeling like using a formal service app. Overall, there's still no simple,

focused, and truly local solution that people can actually trust — something built specifically for a neighborhood or a campus community.

1.4.2 A Look at Existing Platforms

a. Nextdoor

Nextdoor is a local social platform that connects people living in the same area. Users can share news, ask for help, and recommend local services. To keep things trustworthy, it asks users to verify their address.

That said, the platform mixes a lot of different types of content — posts, alerts, ads, recommendations — which can make the interface feel cluttered and hard to navigate. Finding a specific help request among everything else isn't always straightforward.



Figure 1.2. Nextdoor Logo

b. PickMyTask

PickMyTask is a local task-sharing app that connects people who need help with small daily tasks to nearby helpers. It focuses on quick, paid jobs like running errands, buying groceries, or helping with light moving. The flow is simple: post a task with a budget, and helpers can accept or make a counter-offer.

The downside is that the whole model is centered around payment. That makes it a poor fit for casual, unpaid help between neighbors or students who just want to support each other without money getting in the way.



Figure 1.3. PickMyTask Logo

1.4.3 Comparing the Two Platforms

The table below summarizes the key differences in features between Nextdoor and PickMyTask.

Functionalities	Nextdoor	PickMyTask
Neighborhood-based user grouping	Yes	No
Task posting for small errands	Yes	Yes
In-app messaging between users	Yes	Yes
Payment processing for tasks	No	Yes
Paid advertising / local ads	Yes	Yes
Formal helper profile / ratings	No	Yes

Table 1.1. Comparison of functionalities

1.4.4 Our Proposed Solution

To get past the limitations of both Nextdoor and PickMyTask, LocalFix offers a clean, focused web application built around local help within a neighborhood or campus. Unlike Nextdoor, there are no social feeds or ads cluttering the experience — just tasks. And unlike PickMyTask, LocalFix supports both paid help and free peer exchanges, making it much more suitable for everyday situations.

Our solution is built around three core ideas:

- **Hyperlocal filtering:** Users select their neighborhood or campus, and the app only shows tasks and helpers in that specific area. This builds trust and keeps things relevant.
- **Minimal, task-first design:** A clean task board, simple forms, and in-app messaging make it easy to find or offer help without getting lost in menus or distracted by ads.
- **Flexible exchange model:** LocalFix supports both paid services and informal peer exchanges — whether it's homework help, moving boxes, or running an errand. No forced payment model.

By combining the local-community focus of Nextdoor with the task-matching efficiency of PickMyTask — but without the clutter and payment-heavy models — LocalFix offers something lightweight and user-friendly that neither app fully delivers.

1.5 Project Management Methodology

There are lots of project management frameworks out there, each with its own pros and cons. For LocalFix, we went with the **SCRUM** framework because of how flexible it is and how strongly it focuses on delivering a quality product.

1.5.1 The SCRUM Framework

We chose the agile approach, and SCRUM in particular, because it lets us break the work down into small, manageable chunks. This keeps the project organized, improves how productive the team is, and makes it much easier to respond to changing needs. In our case, we adapted SCRUM to fit our context, using short development cycles while always keeping the user experience front and center.

SCRUM was originally created for software development, but it can honestly be applied to all kinds of projects. The name comes from rugby, where a scrum is a moment of tight teamwork — and that spirit of close collaboration is exactly what SCRUM brings to project management.

The work is organized into short phases called **sprints**, each typically lasting one to four weeks. These cycles let the team plan, track progress, and adjust course regularly throughout the project.

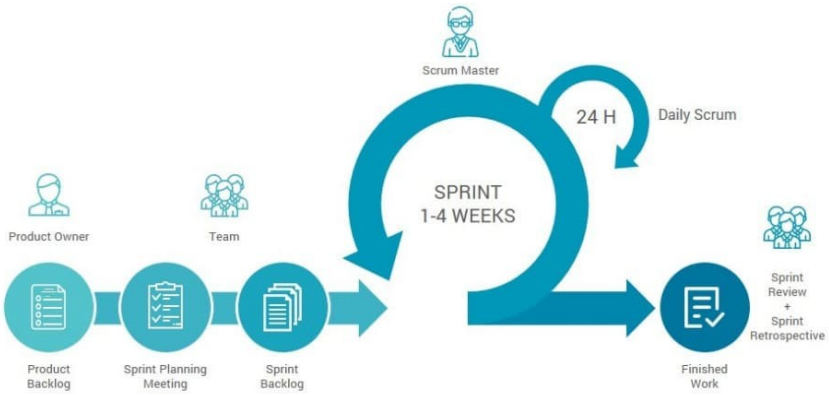


Figure 1.4. SCRUM Process

1.5.2 Roles in SCRUM

The SCRUM team is made up of three key roles:

- **SCRUM Master:** The person responsible for making sure SCRUM is properly followed by the team. They keep things running smoothly and act as a driving force behind the development process.
- **Product Owner:** The person who manages the product backlog and defines what the final product should look like. They act as the bridge between the development team and the end users or client.
- **Development Team:** The group of people actually building the product — developers, designers, testers, and anyone else contributing to the project.

1.5.3 SCRUM Artifacts

SCRUM uses a few key artifacts to help everyone stay on the same page:

- **Product Backlog:** The full list of everything that needs to be built, ordered by priority. It evolves as the project progresses.
- **Sprint Backlog:** The subset of the product backlog that the team commits to working on during a specific sprint.
- **Burn-down Chart:** A visual graph showing how much work is left in the sprint. As tasks are completed, the line drops toward zero, helping the team stay on track.

1.5.4 SCRUM Events

Each sprint in SCRUM revolves around four key events:

- **Sprint Planning:** A meeting at the start of each sprint where the team agrees on the sprint goal and picks the items they'll work on.
- **Daily Scrum:** A short daily check-in (about 15 minutes) where each team member quickly shares what they've done, what they're working on, and if anything is blocking them.
- **Sprint Review:** A meeting at the end of each sprint where the team presents completed work, collects feedback, and closes out the sprint.
- **Sprint Retrospective:** A reflection session (usually around 45 minutes) where the team discusses what went well, what didn't, and how to improve going forward.

1.6 Conclusion

In this chapter, we introduced the overall context of the LocalFix project: the problem we're tackling, the solution we're proposing, and the management approach we're using to get there. The next chapter will dig deeper into the actual requirements of the system — both functional and non-functional — and present them using UML diagrams.

Chapter 2

Analysis of Needs and Specification

2.1 Introduction

This chapter is where we really get into the details of what LocalFix needs to do and how it should behave. We start by identifying the different types of users (actors) involved in the platform, then walk through the functional requirements (what the system should do) and the non-functional requirements (how well it should do it). We also present our work planning using SCRUM, including the product backlog, sprint breakdown, and an overview of the hardware, technologies, and tools we chose for development.

2.2 Project Sprints

Sprint	Description	Duration
0	Training on React, Node.js, and the LocalFix architecture	2 weeks
1	Task management and implementation of authentication (login, roles, route guards)	1 week
2	Development of messaging, user reporting, and profile management	2 weeks
3	Implementation of the interactive map, notifications, and admin tools	2 weeks
4	Development of the rating and review system, and general polish	2 weeks

Table 2.1. The sprints of the LocalFix project

2.3 Analysis of Needs and Specification

In this section, we identify and specify the needs that our application has to meet, covering all the main functionalities we plan to implement.

2.3.1 Identifying the Actors

An actor represents a role played by someone who interacts with the system. For LocalFix, we've identified three main actors:

- **Help Requester:** Can create, read, and update their own task requests, and view the profiles and availability of nearby helpers.
- **Helper:** Can browse available tasks, accept or decline requests, update the status of ongoing tasks, and communicate with the requester through the messaging system.
- **Administrator:** Manages user accounts, monitors interactions, and steps in when there's inappropriate behavior or a dispute between users, keeping the platform safe and reliable.

2.3.2 Functional Requirements

A functional requirement is something the system must be able to do. Here's what each actor needs from LocalFix:

Help Requester:

- Authenticate to the application
- Create a new task (title, description, category, location)
- View the list of available nearby tasks
- Modify or delete their own tasks
- Accept or reject a helper's proposal
- Send and receive messages with a helper

Helper:

- Authenticate to the application
- Browse nearby tasks available for helping
- Propose to help on a specific task
- Send and receive messages with a requester

Administrator:

- Manage user accounts (activate, deactivate, or remove)
- View general statistics and platform activity
- Handle reported content or disputes between users

Figure 2.1 shows the overall use case diagram of our application.

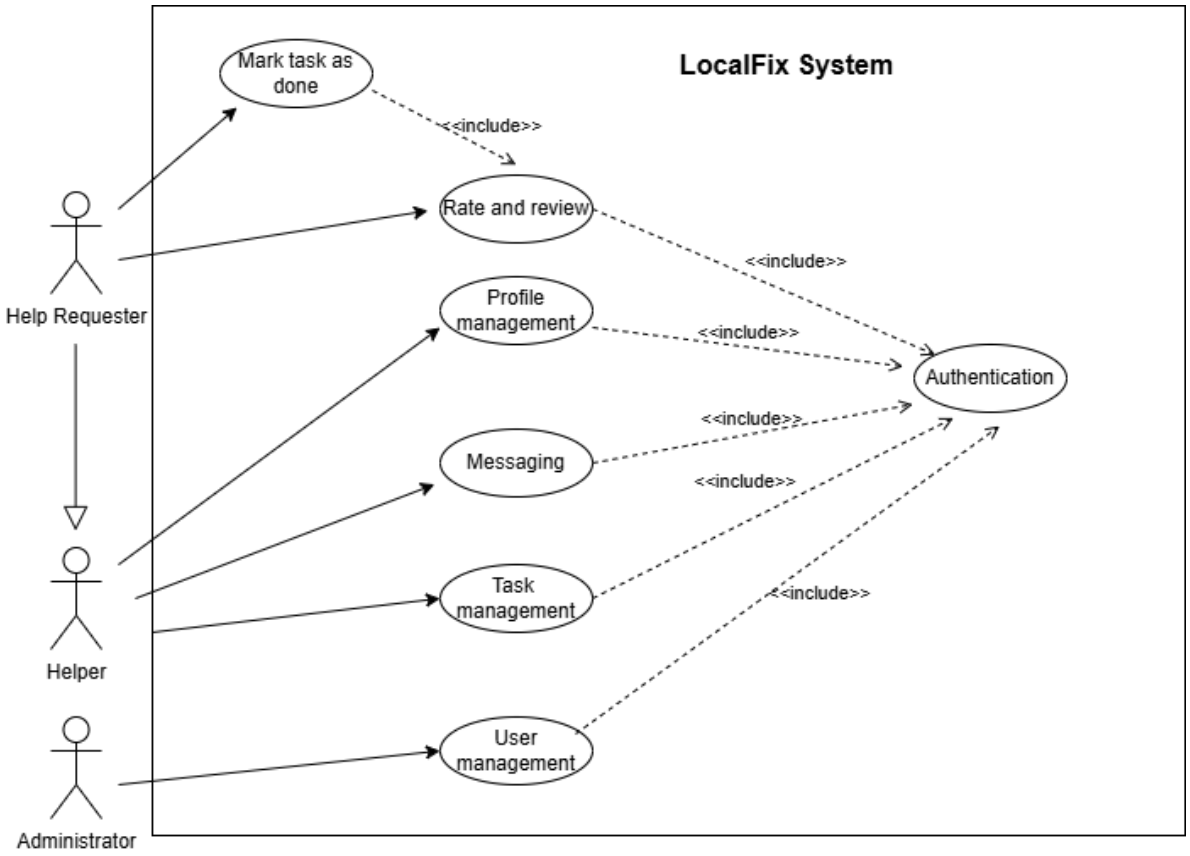


Figure 2.1. Global use case diagram

2.3.3 Non-Functional Requirements

Non-functional requirements don't describe what the system does, but how it does it. They define the quality constraints that LocalFix must meet:

- **Reliability:** The platform must be stable, resistant to failures, and available to users at all times.
- **Usability:** The interface must be intuitive, easy to navigate, and accessible to all users, including those with specific accessibility needs.
- **Security:** The application must handle authentication, authorization, and data encryption properly, with monitoring and threat detection in place.
- **Simplicity:** The interface shouldn't be overwhelming. Navigation between pages must be smooth, without overloading the user with too many options at once.
- **Performance:** The system must handle a high volume of requests while maintaining fast response times, even during peak usage.
- **Extensibility:** The architecture must be easy to maintain and evolve, allowing features to be added, updated, or removed without breaking everything else.

2.3.4 Work Planning with SCRUM

Before kicking off the first sprint, we put together a **product backlog** — a prioritized list of all the features and tasks we want to build, based on what users actually need. The product backlog acts as the central guide for the entire project and gets updated as requirements evolve, feedback comes in, or priorities shift.

2.4 Composition of the Product Backlog

The product backlog is managed by the Product Owner and contains all the features the team needs to build. Each item is written as a **User Story** and is described using:

- **ID:** A unique identifier for the user story.
- **Feature:** The feature the story belongs to.
- **User Story:** A short description of what the user wants and why.
- **Priority:** How important this feature is.
- **Complexity:** How difficult it is to implement.

Table 2.2. LocalFix product backlog

ID	Feature	User Story	Priority	Complexity
1	Login	As a user, I want to log in so I can access my account based on my role.	High	Medium
2	Create tasks	As a Help Requester, I want to create help requests so I can ask for help.	High	High
3	Browse tasks	As a Helper, I want to see nearby tasks so I can choose what I want to do.	High	Medium
4	Messages	As a user, I want to send and receive messages to communicate about a task.	Medium	Medium
5	Profile management	As a user, I want to edit my profile so people know who I am.	Medium	Medium
6	Reporting	As a user, I want to report other users for any offenses.	Medium	Medium
7	Task status	As a user, I want to see if a task is posted, accepted, in progress, or done.	Medium	Low
8	Admin tools	As an Admin, I want to manage users and tasks to keep the app safe.	Medium	High
9	Notifications	As a user, I want notifications when someone replies or accepts my task.	High	Medium

2.4.1 Sprints Breakdown

Release 1 — MVP (Minimum Viable Product): A working LocalFix app with the core features a user can actually use.

- **Sprint 1 — Core setup and login:** Create user accounts, implement login and basic profile (name, basic info), implement task management tools (create task, accept task, edit task, view task, delete task) and set up the project structure and database schema.
- **Sprint 2 — Messaging and user setup:** Implement in-app messaging between the helper and the help requester, implement user reporting for safety and profile management tools for more customization (display name, bio, skills).

Release 2 — Extra features and polish: Adding more useful functionality and improving the overall experience.

- **Sprint 3 — UX core setup:** Implement admin tools (account suspension, report verification, task register and user overview), implement in-app notifications for received and broadcasted messages, implement discovery map for visual helper filtering based on location, skill and rating, implement display task status (Open, In-Progress, Closed).
- **Sprint 4 — Rating and polishing:** Implement "Rate and Review Helper" after a task is completed, fix bugs and polish the UI and UX.

2.5 Application Architecture

2.5.1 Logical Architecture

For LocalFix, we went with the **MVC architecture** (Model–View–Controller), which divides the application into three clearly separated parts:

- **Model:** Handles all the data — tasks, users, ratings — fetching and organizing it from the database for the controllers.
- **View:** The part the user actually sees: task browsing pages, task creation forms, profiles. It takes data from the Model and displays it cleanly.
- **Controller:** The middle layer that receives user actions, applies the application logic, and decides what to send back to the View.

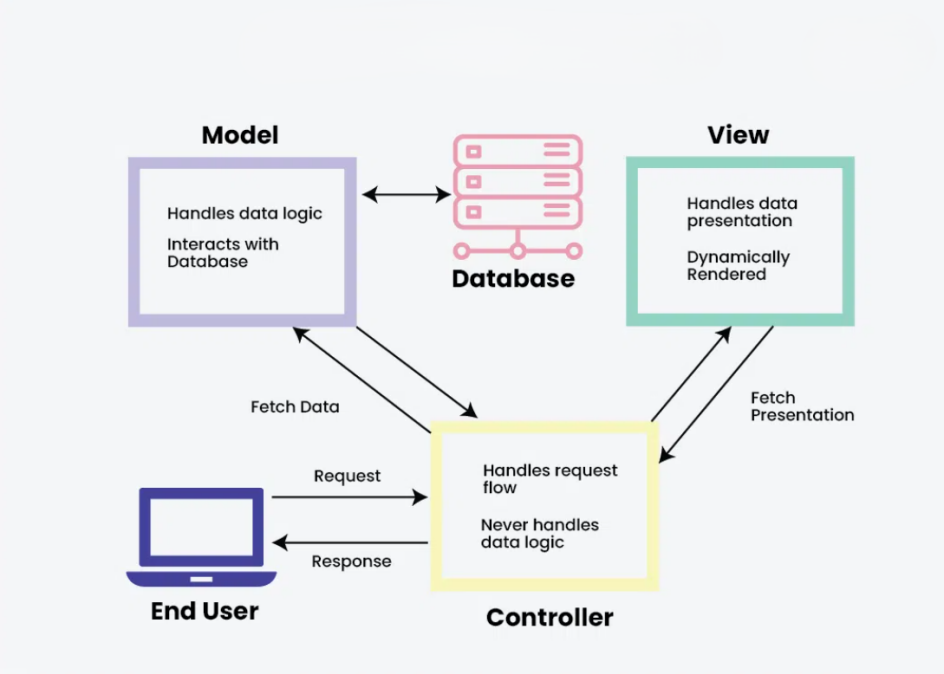


Figure 2.2. MVC Architecture

On top of the MVC structure, we also use an **event-driven architecture** for internal communication. Instead of components calling each other directly, they produce events when something happens — like a user action or a data change — and other components listen and react. This keeps everything loosely coupled and makes it much easier to add features like notifications and real-time updates.

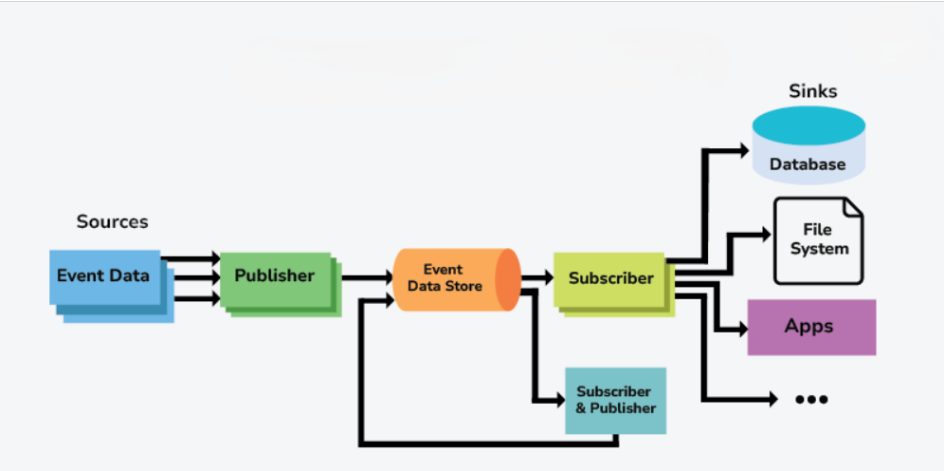


Figure 2.3. Event-Driven Architecture

2.5.2 Physical Architecture

For the physical architecture, we chose the classic **three-tier architecture**: client side, server side, and database. It's a well-proven choice that gives us a good balance of simplicity, scalability, and security.

- **Presentation layer:** The user interface — everything the user sees and interacts with, built with HTML, CSS, and ReactJS.
- **Application layer:** The server-side logic — handling tasks, users, messages, and database calls. This is where all the business rules live.
- **Data layer:** The database that stores all of LocalFix's data — users, tasks, messages, ratings. All managed through a DBMS (Database Management System).



Figure 2.4. 3-Tier Web Architecture

2.6 Software Environment of the Project

Here's a rundown of the main tools and technologies we used to build and manage LocalFix.

2.6.1 Project Management Tools

- **Microsoft Teams:** Our main communication tool for chats, video calls, and file sharing.

Figure 2.5. Microsoft Teams Logo

- **Jira:** Used to organize tasks, issues, and sprints throughout the project.

Figure 2.6. Jira Logo

2.6.2 Development and Testing Tools

- **Node.js:** A JavaScript runtime used for the server-side of LocalFix, enabling fast and scalable backend logic.

Figure 2.7. Node.js Logo

- **ReactJS:** The JavaScript library we used to build the dynamic, interactive user interface of LocalFix.

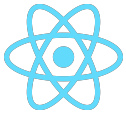


Figure 2.8. ReactJS Logo

- **VS Code:** Our code editor of choice — lightweight, powerful, and free.



Figure 2.9. VS Code Logo

- **MySQL:** A relational database system used to store structured data like tasks and users.



Figure 2.10. MySQL Logo

- **MongoDB:** A NoSQL database used for more flexible, document-based data storage.



Figure 2.11. MongoDB Logo

- **TeXstudio:** The LaTeX editor we used to write this very report.



Figure 2.12. TeXstudio Logo

- **Draw.io:** A web-based tool we used to create all our diagrams and architecture visuals.



Figure 2.13. Draw.io Logo

2.7 Conclusion

In this chapter, we mapped out the full plan for LocalFix: who the actors are, what the system needs to do, how it should behave, and how the work is organized into sprints. We also defined the logical and physical architecture, and presented all the tools and technologies we're using. In the next chapter, we'll dive into the first release and walk through what was built in the first two sprints.

Chapter 3

Release 1

3.1 Introduction

This chapter covers the work done during the first release of LocalFix. We walk through the two sprints that make up Release 1: the first focused on getting users set up and managing tasks, and the second on communication, reporting, and profile management. For each sprint, we present the objectives, the backlog, the use case analysis, the class and sequence diagrams, and a look at the interfaces we built.

3.2 Sprint 1: Task and User Management

3.2.1 Sprint 1 Objective

The goal of the first sprint was to build the core of LocalFix: user authentication and the main task management features. This means users can log in, create tasks, edit or delete their own tasks, and browse what's available.

ID	Feature	User Story	Priority	Complexity
1	Login	As a user, I want to log in so I can access my account and my tasks.	High	Low
2	Create Task	As a Help Requester, I want to create a task so I can ask for help.	High	High
3	Edit Task	As a Help Requester, I want to edit a pending task I've already posted.	High	Medium
4	Delete Task	As a Help Requester, I want to delete a task that hasn't been accepted yet.	Medium	Medium
5	View Task List	As a user, I want to see the list of available tasks so I can choose what to do.	Medium	Medium

Table 3.1. Product Backlog for Sprint 1

3.2.2 Analysis of Sprint 1

Figure 3.1 shows the use-case diagram for Sprint 1, illustrating how users interact with the system during the implementation of the core task management features.

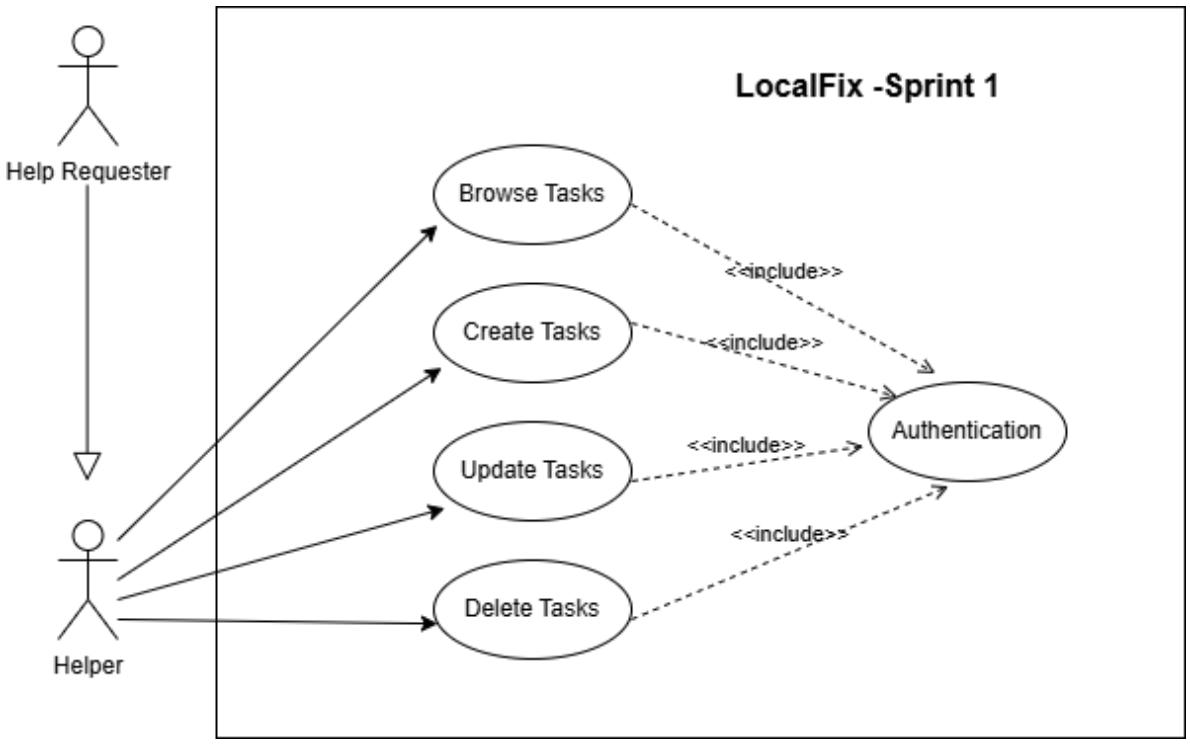


Figure 3.1. Sprint 1 Use Case Diagram

Description of Scenarios:

Authentication

1. The user enters their login and password.
2. The user clicks “Login”.
3. If the credentials are correct, the system redirects to the dashboard.
4. If not, an error message appears and the user can try again.

Create Task

1. The user opens the “Create Task” page.
2. The user fills in the task details: title, description, category, and location.
3. The user can go back and refine the description before submitting.
4. The user clicks “Submit” to publish the task.

Task Edit

1. The user opens the task list.
2. The user clicks on the “Edit” button next to the task.
3. The user changes any field they want.
4. The user clicks “Submit” to republish the updated task.

Task Delete

1. The user opens the task list.
2. The user clicks on the “Delete” button next to the task.

3.2.3 Design of Sprint 1

a. Class Diagram of Sprint 1

The main entities in the task management module are: **User**, **Task**, **Category**, **Tag**, **Message**, and **Rating**.

- A **User** has attributes like id, name, email, phone number, skills, and rating. A user can create and manage zero or more tasks.
- A **Task** has attributes like id, userId (the requester), helperId (the helper who accepted), title, description, category, price (if paid), location, status (posted / accepted / in progress / done), start and end dates, and timestamps. Each task is created by exactly one requester and can be accepted by one helper.
- A **Category** has a title and description. Categories can contain tasks and subcategories (e.g., "Homework Help" might include "Math", "Programming", etc.).
- A **Tag** has id, name, and timestamps. Tags can be associated with tasks to help with filtering (e.g., "urgent", "online", "on-site").

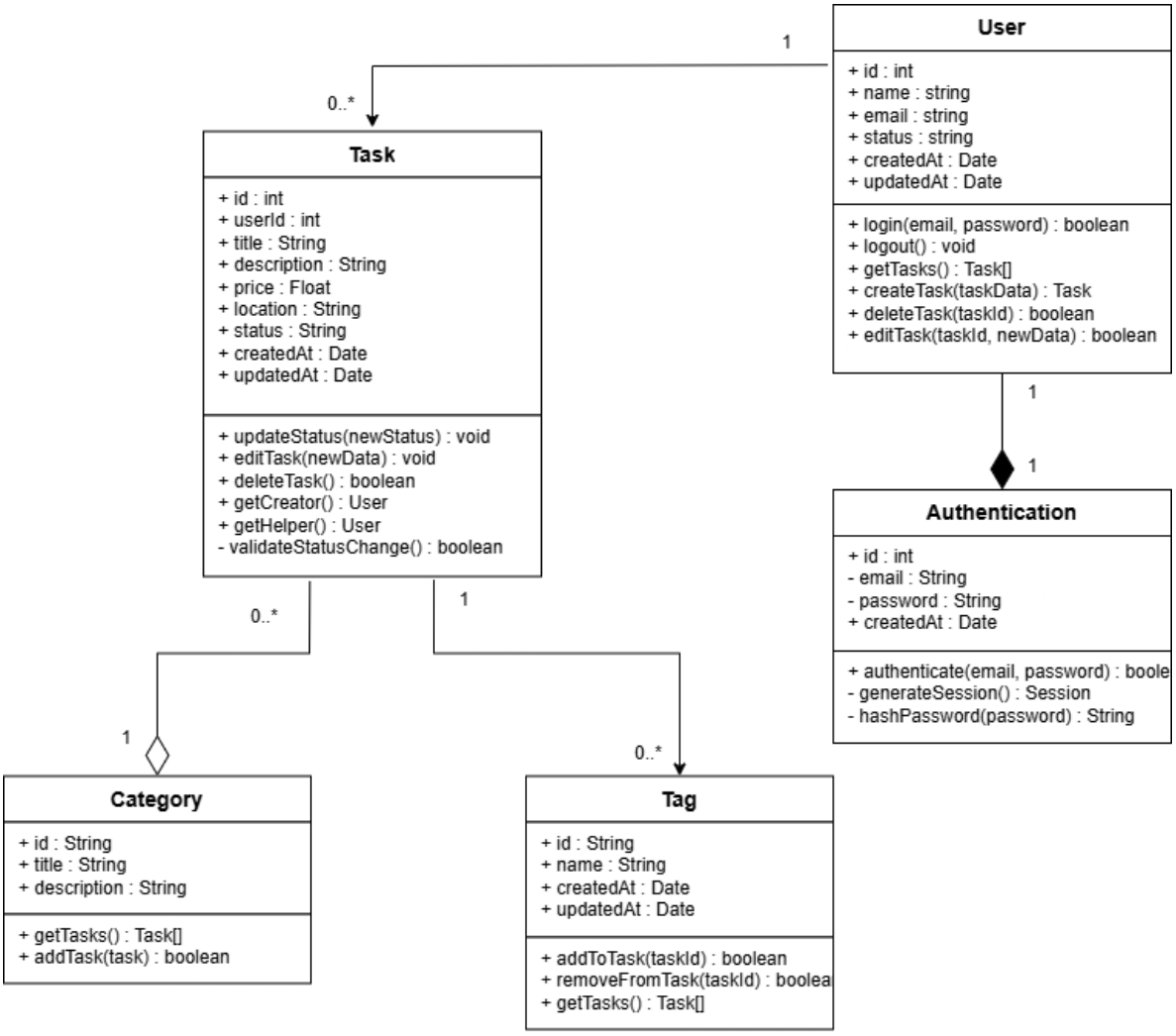


Figure 3.2. Sprint 1 Class Diagram

b. Sprint 1 Sequence Diagrams

Figure 3.3 shows the sequence diagram for the task creation use case.

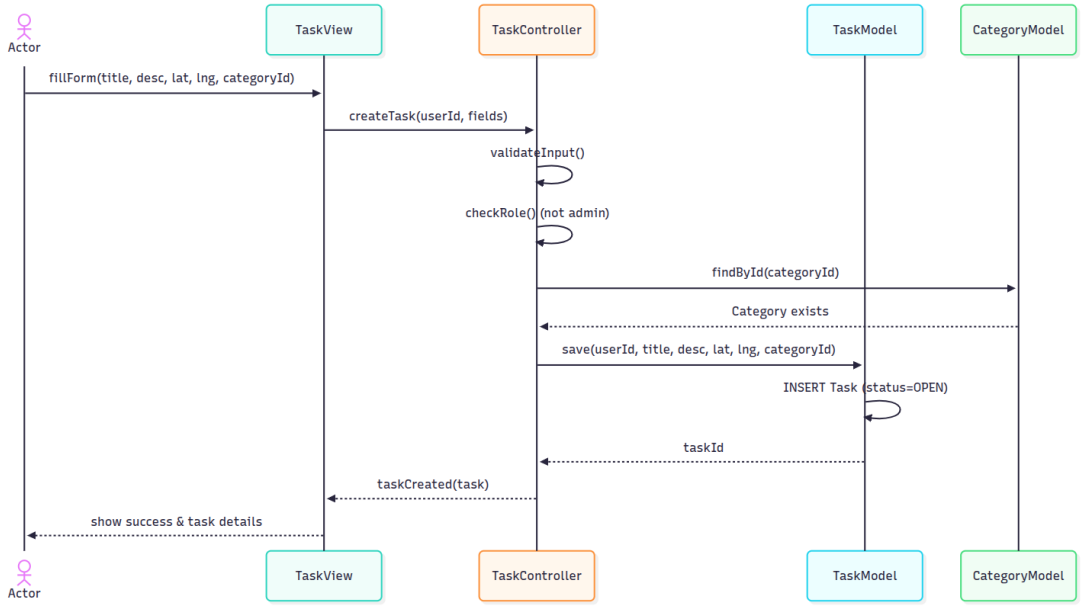


Figure 3.3. Sprint 1 Create Task Sequence Diagram

3.2.4 Completion of Sprint 1

To give a clearer picture of what was built, here are the main interfaces developed during Sprint 1.

Authentication Interface:

Figures 3.4 and 3.5 show the login and registration forms that allow users to access their account based on their role.

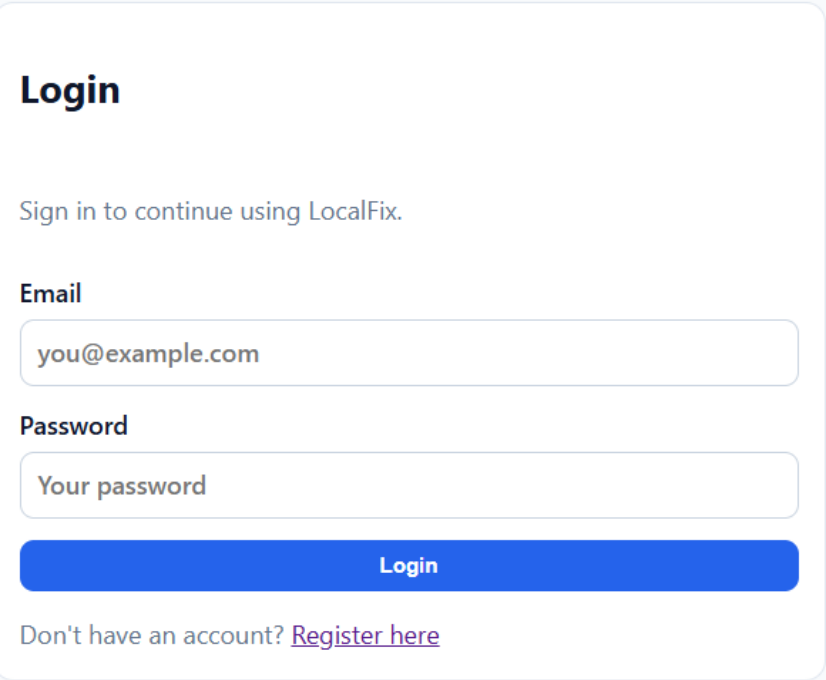


Figure 3.4. Authentication Interface

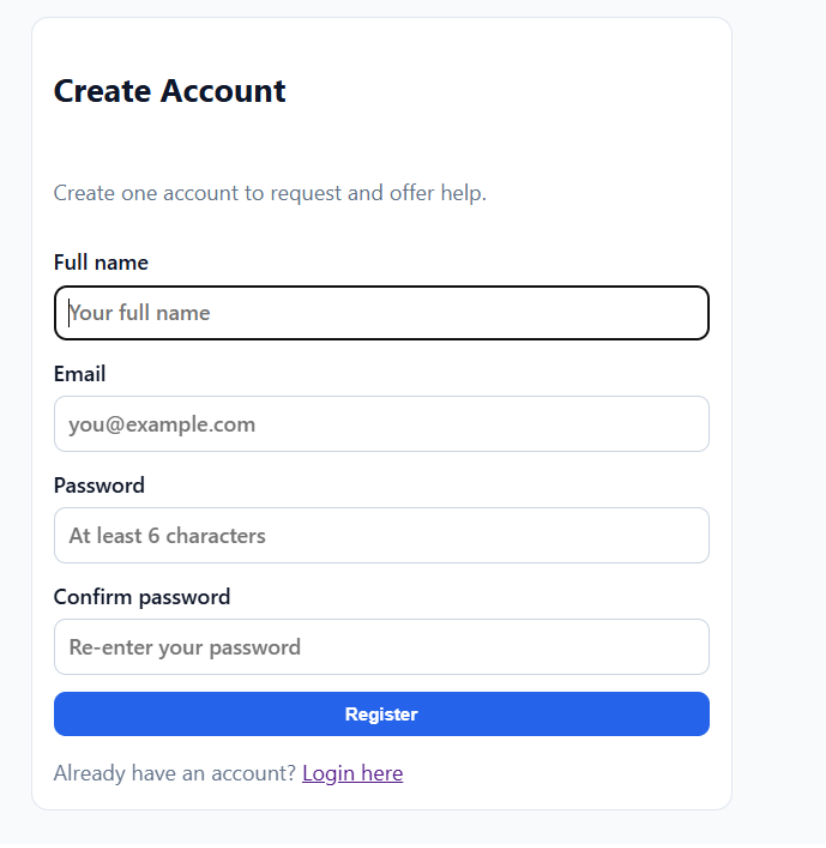


Figure 3.5. Register Interface

Task Management:

Figures 3.6, 3.7 and 3.8 represent the task creation, task delete/edit, and the view task interfaces respectively.

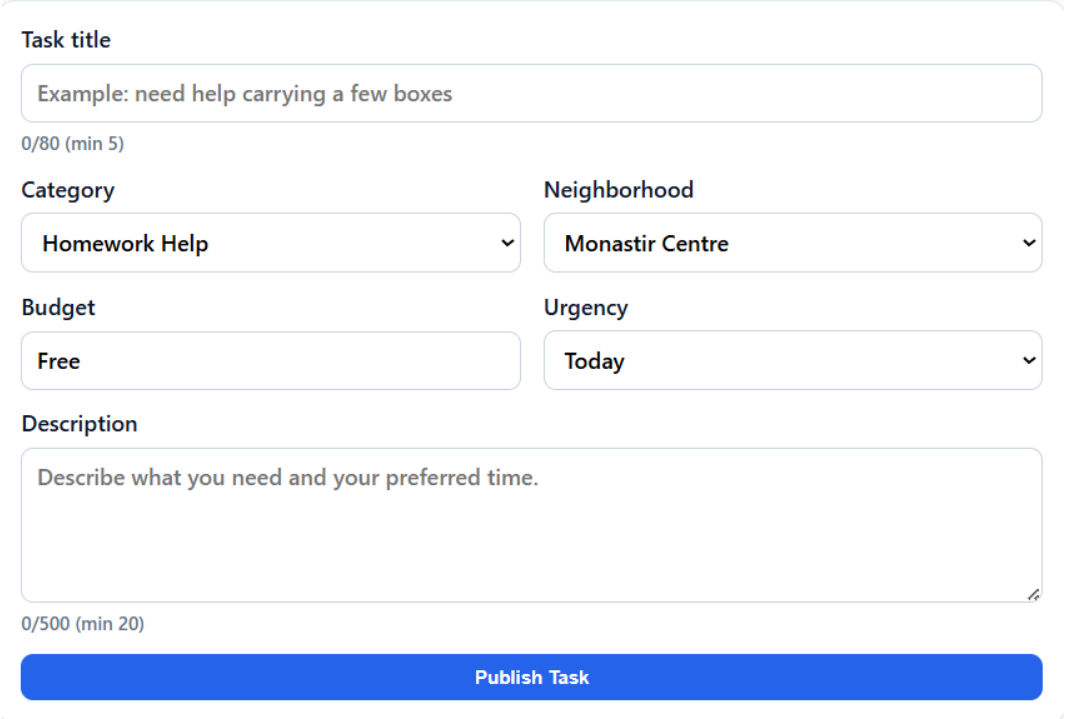


Figure 3.6. Post-Task Interface

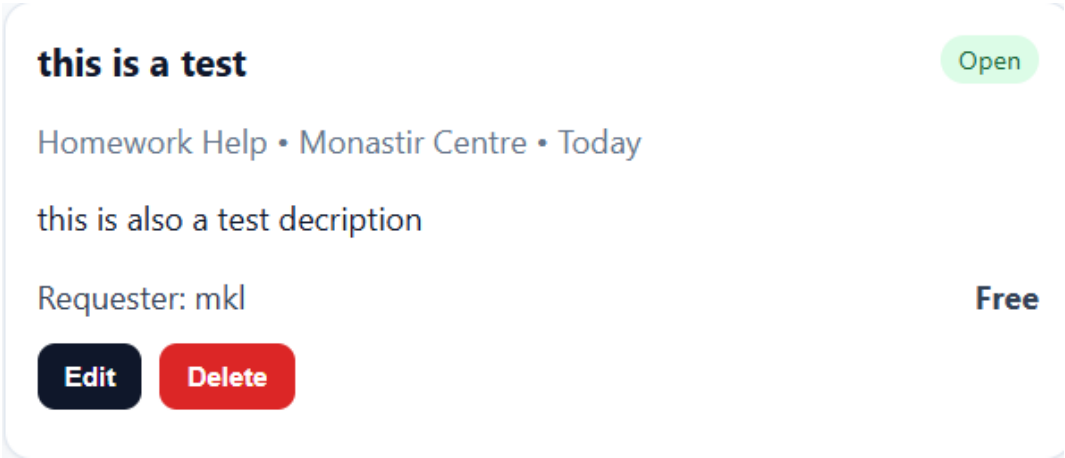


Figure 3.7. Task Edit/Delete Interface

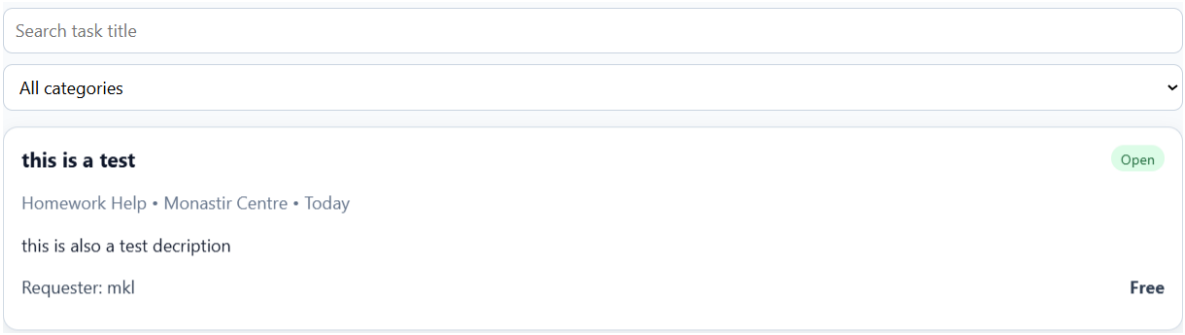


Figure 3.8. Task View Interface

3.3 Sprint 2: User Management

3.3.1 Objective of Sprint 2

The second sprint focused on three modules: **Messaging**, **Reporting**, and **Profile Management**. These features allow users to communicate through in-app messages, report inappropriate behavior, and manage their personal profiles.

3.3.2 Backlog of Sprint 2

Id	Feature	User Story	Priority	Complexity
1	Authentication	As a user, I want to authenticate so I can access my personalized space.	High	Low
2	Messages	As a user, I want to send and receive messages about a task.	Medium	Low
3	Report Helper	As a Help Requester, I want to report a Helper so the platform can review their behavior.	High	Medium
4	Profile Management	As a user, I want to update my profile (name, skills, availability) to better reflect who I am.	High	Medium

Table 3.2. Sprint 2 Product Backlog

3.3.3 Analysis of Sprint 2

Figure 3.9 presents the use case diagram for Sprint 2, covering user messaging, reporting, and profile management.

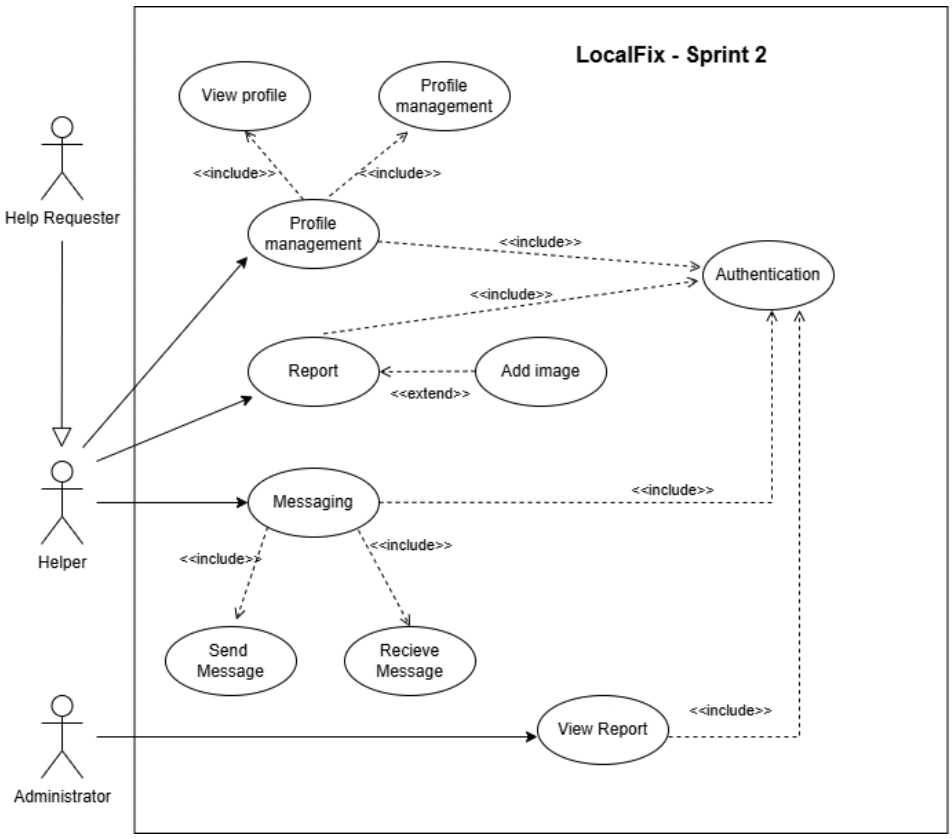


Figure 3.9. Sprint 2 Use Case Diagram

Description of Scenarios:

Send Message

1. The user opens an active task or report from the dashboard.
2. The user clicks “Message” to open the chat.
3. The user types their message and clicks “Send”.
4. If the message is empty, the user is returned to step 3.

Submit User Report

1. The user navigates to the “Report” section from the dashboard.
2. The user fills in the details: who to report, the reason, and a description.
3. The user clicks “Submit Report” for the admin to review.

Profile Management

1. The user clicks “Profile” from the dashboard menu.
2. The user can update their display name and password.
3. The user can view the status of their current tasks.
4. The user can see their current and past reports.

3.3.4 Design of Sprint 2

a. Class Diagram of Sprint 2

The main entities in Sprint 2 are: **User**, **Profile**, **Report**, **Chat**, **Message**, and **Notification**.

- A **Profile** has a displayName and password, along with methods for updating the profile and retrieving user stats.
- A **Report** has an id, title, and description, and represents a complaint submitted against another user.
- A **Chat** has an id and contains multiple messages, enabling real-time communication between a requester and a helper.
- A **Message** has an id, content, timestamp, and isRead flag, representing individual messages within a chat.

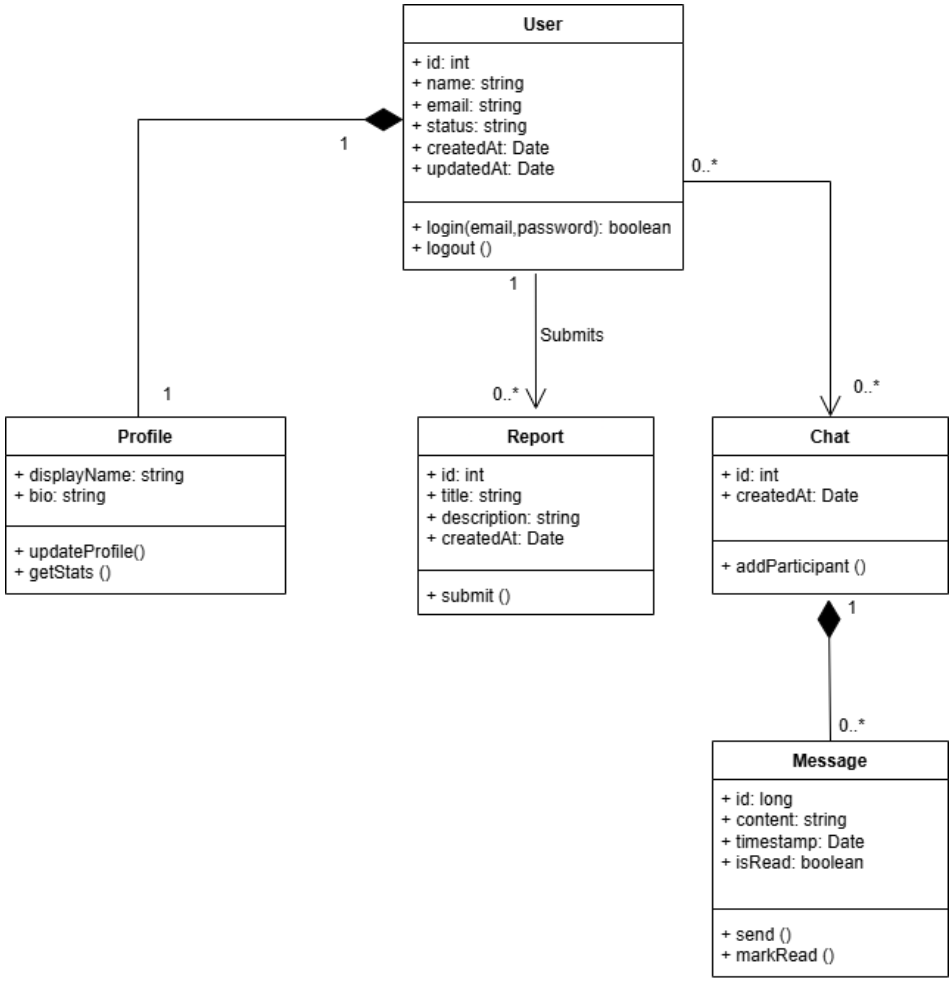


Figure 3.10. Sprint 2 Class Diagram

b. Sprint 2 Sequence Diagrams

Figure 3.11 shows the sequence diagram for the Report use case.

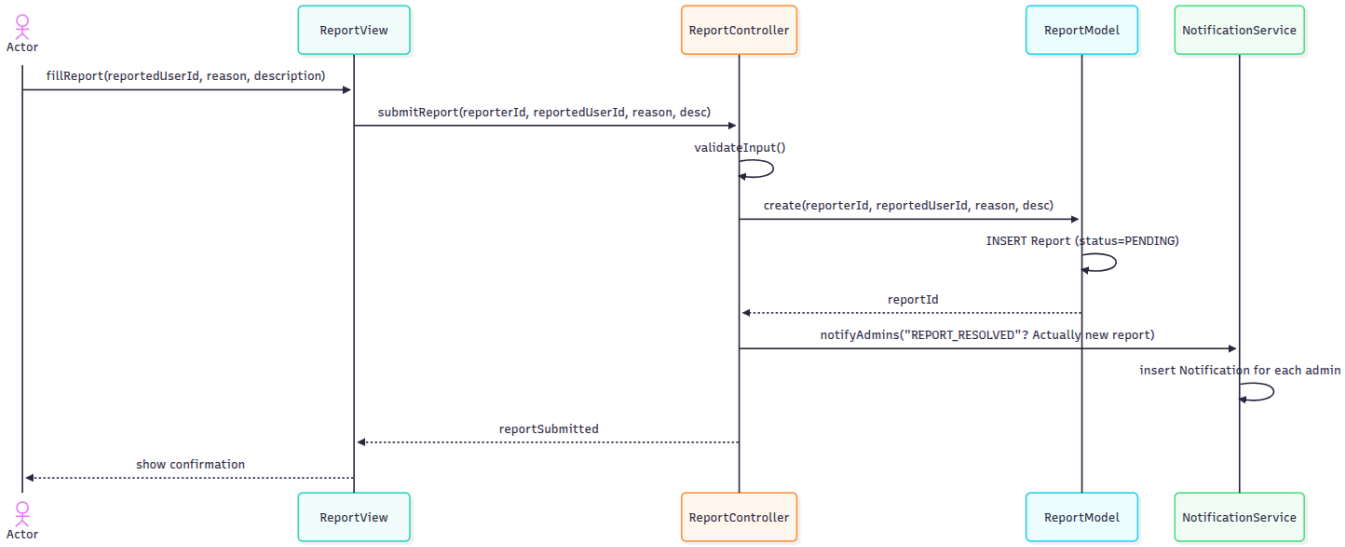


Figure 3.11. Sprint 2 Report Sequence Diagram

3.3.5 Completion of Sprint 2

Here are the main interfaces built during Sprint 2.

Messaging Interface:

Figure 3.12 shows the messaging interface that connects helpers and requesters through in-app chat.

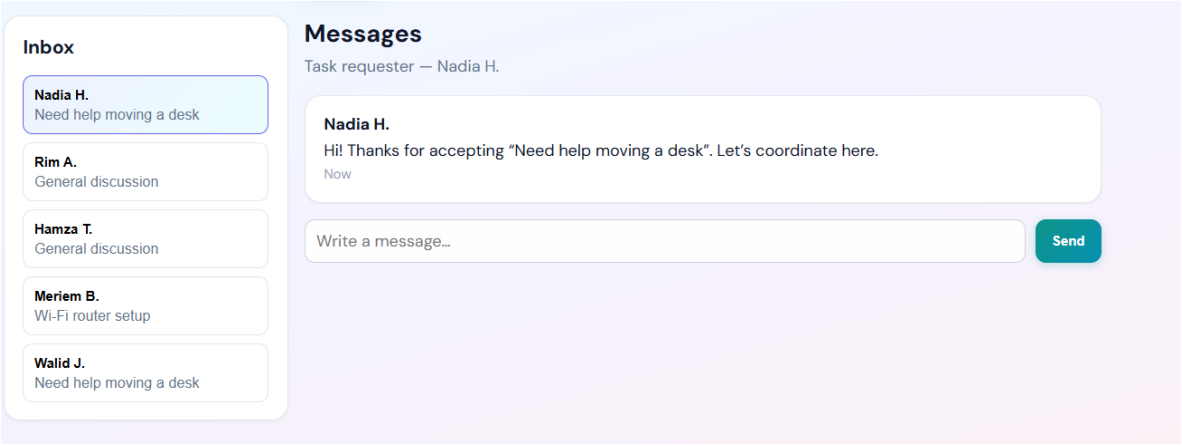


Figure 3.12. Messaging Interface

Reporting Interface:

Figure 3.13 shows the reporting interface, which allows users to flag inappropriate behavior.

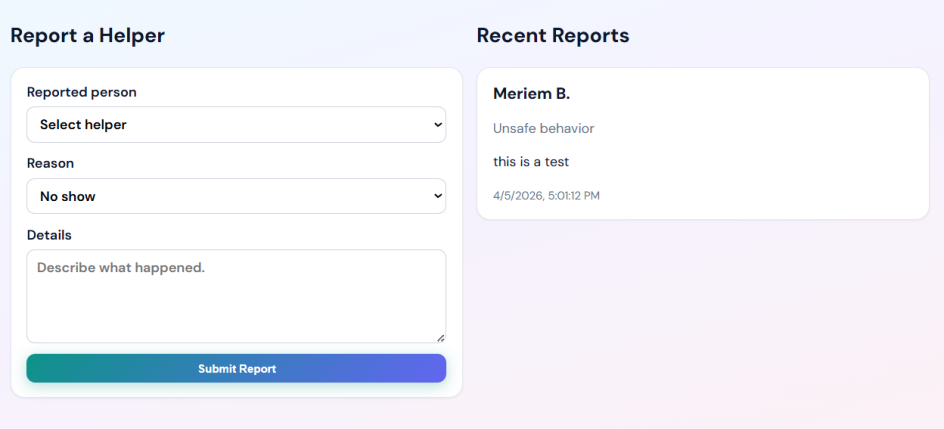


Figure 3.13. Reporting Interface

Profile Management Interface:

Figure 3.14 shows the profile page where users can update their personal information.

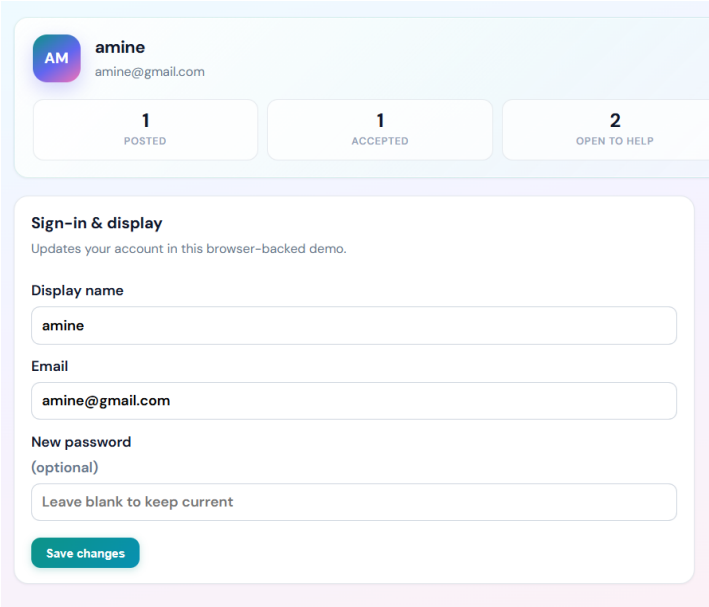


Figure 3.14. Profile Management Interface

3.4 Conclusion

Sprint 2 is done. We now have messaging, reporting, and profile management up and running. In the next chapter, we move into Release 2, which focuses on connecting all these pieces together with task assignment workflows, real-time status tracking, notifications, and admin tools.

Chapter 4

Release 2

4.1 Introduction

Chapter 4 covers the second release of LocalFix, where we build on everything from Release 1 and add the more advanced features that bring the platform to life. This includes task state management (open → in-progress → closed), a map-based discovery view, an admin dashboard, a full notification system, and the rating and review module. We also took time during this release to clean up and polish the features we had already built.

4.2 Sprint 3: Interaction, Task Status, and Platform Visibility

4.2.1 Objective of Sprint 3

The third sprint was all about connecting the dots. The task objects from Sprint 1 and the communication channels from Sprint 2 needed to work together: helpers commit to open tasks, both sides track progress through explicit status changes, requesters close tasks when the work is done, and users receive notifications about important events. On the admin side, we added visibility over the live task register and account list so the platform can be properly supervised.

4.2.2 Backlog of Sprint 3

ID	Feature	User Story	Priority	Complexity
1	Task status visibility	As a user, I want to see whether a task is open, in progress, or completed.	High	Low
2	In-app notifications	As a user, I want notifications when someone accepts my task, messages me, or when the admin sends an update.	High	Medium
3	Map and discovery	As a member, I want to browse helpers on a map with filters to find someone nearby.	Medium	Medium
4	Report moderation	As an Admin, I want to assign, verify, and resolve reports with notes so trust-and-safety issues are tracked transparently.	High	High
5	Admin dashboard	As an Admin, I want to see aggregate stats and tables of tasks and accounts so I can supervise the platform.	Medium	Medium

Table 4.1. Sprint 3 Product Backlog

4.2.3 Analysis of Sprint 3

Figure 4.1 presents the use-case diagram for Sprint 3, showing how the Help Requester, Helper, and Administrator interact with the new features.

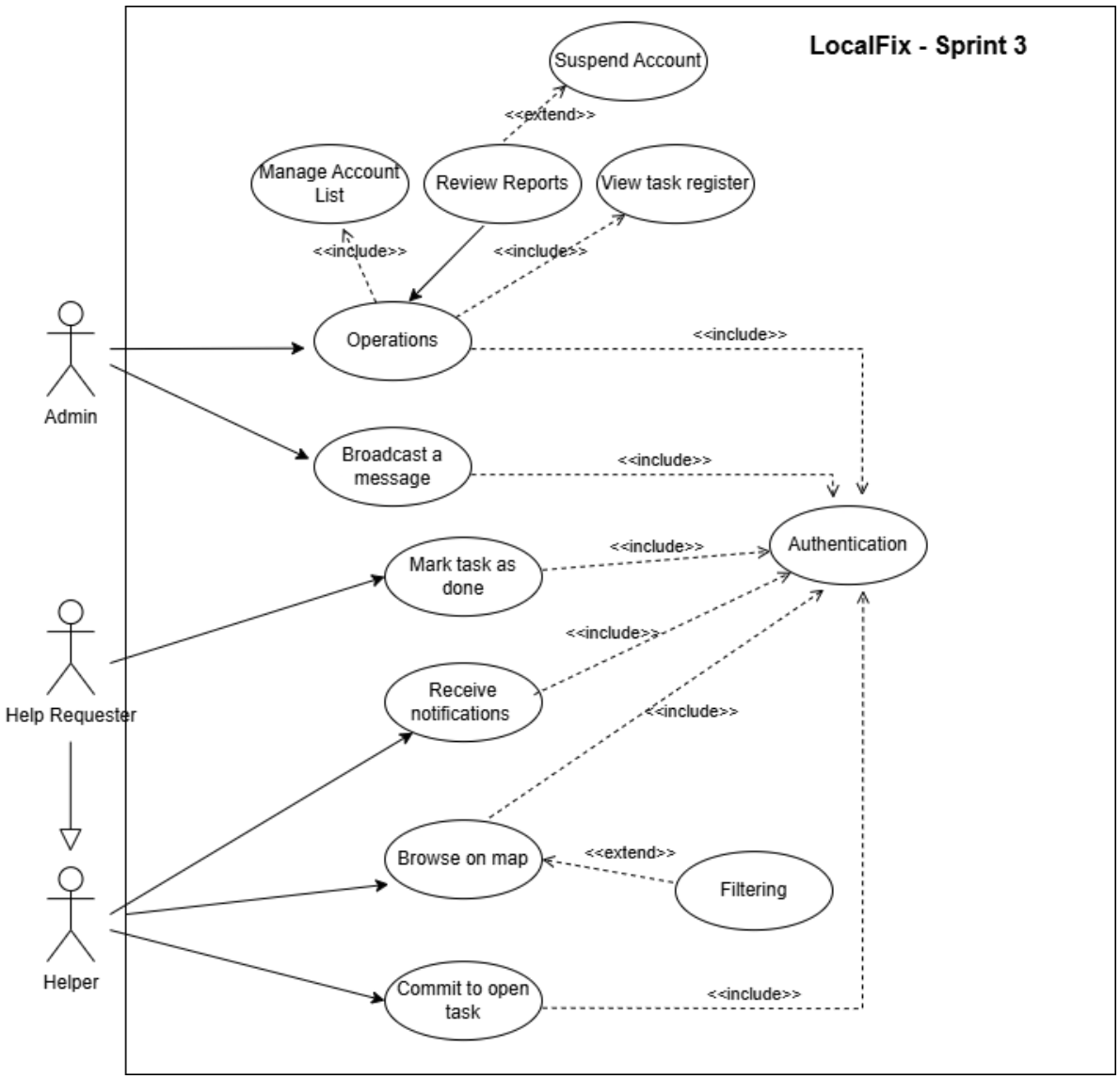


Figure 4.1. Sprint 3 Use Case Diagram

Description of Scenarios:

Mark Task Complete

1. The requester opens an in-progress task they own.
2. The requester selects “Mark complete”.
3. The system validates that the task lifecycle allows completion, updates the status to *completed*, and notifies the helper.
4. The interface may prompt for a rating (covered in Sprint 4).

Consult Notifications

1. The user opens the notification view from the main navigation.
2. The system lists events targeted to the signed-in user (task accepted, message received, admin broadcast, new report, etc.).
3. The user marks items as read; unread counts appear in the header until cleared.

Moderate a Report (Administrator)

1. The administrator opens the console and reviews the report queue.
2. The administrator assigns a moderator name, toggles verification, or resolves the case with a resolution note.
3. The system stores the updated status and keeps the record for audit purposes.

4.2.4 Design of Sprint 3

a. Class Diagram of Sprint 3

Sprint 3 extends the data model built in the previous sprints. The Task aggregate now supports explicit state transitions; the User model differentiates regular users from administrators through a role flag; Report entities include moderation fields (status, assignee, resolution note, verification flag); Notification entities carry user-targeted messages with read state and timestamps; and the map view reuses Helper profile data with geospatial attributes.

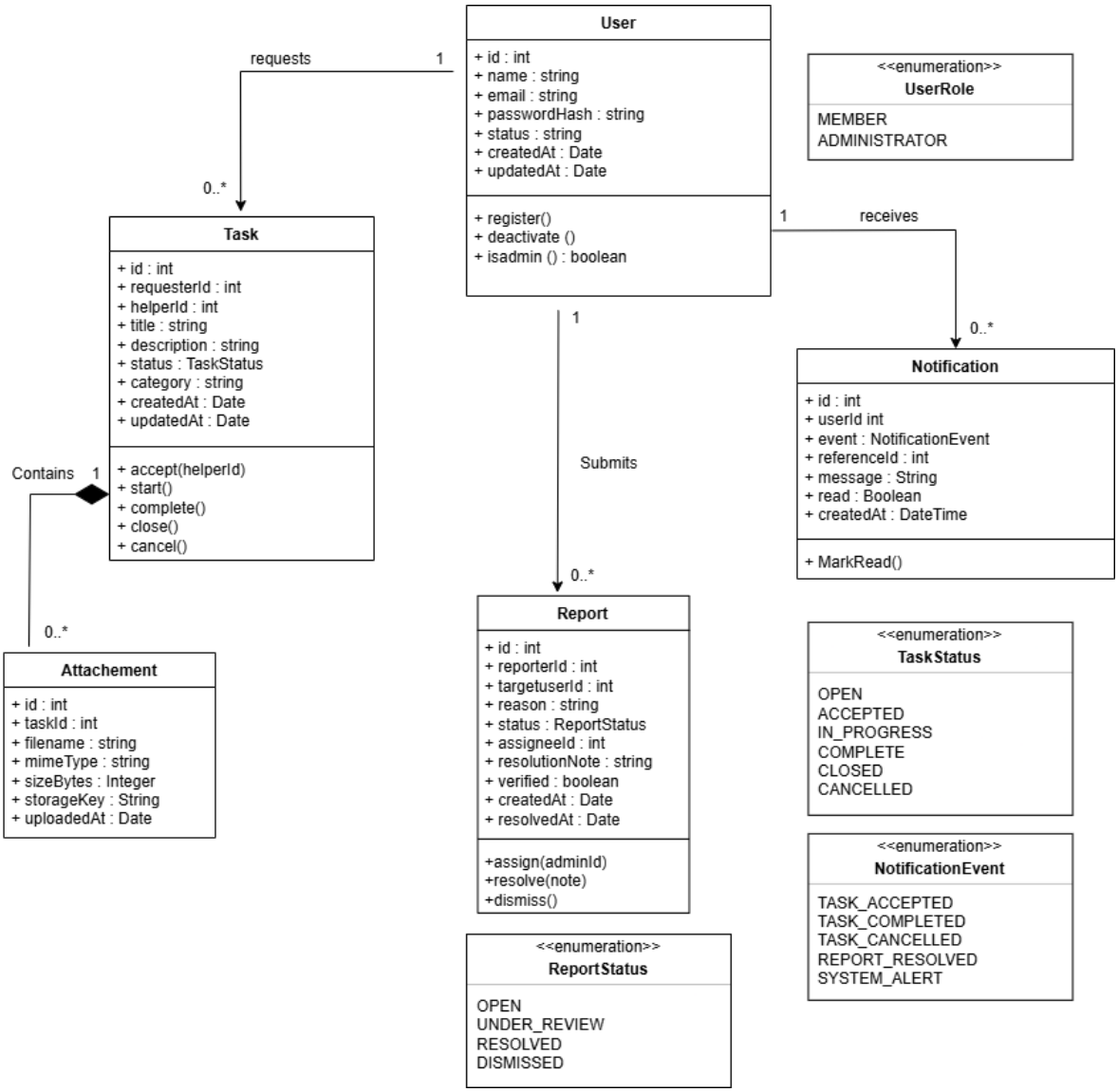


Figure 4.2. Sprint 3 Class Diagram

b. Sprint 3 Sequence Diagram

Figure 4.3 summarizes the administrator moderation flow for a submitted report.

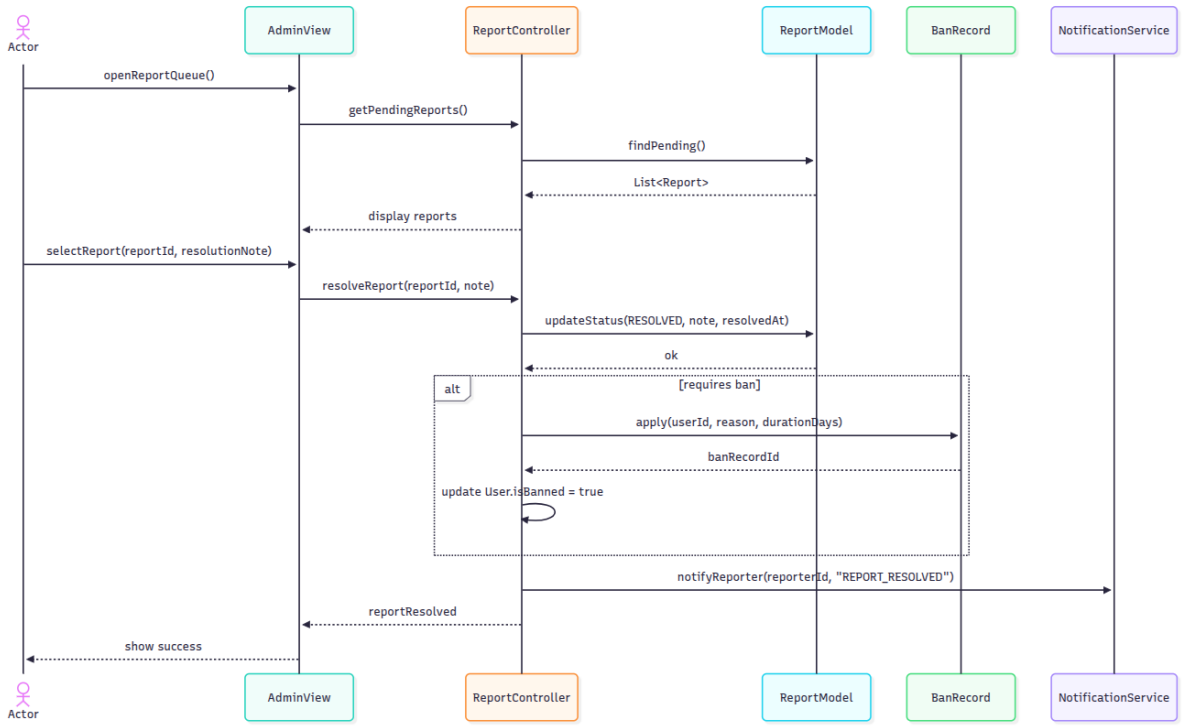


Figure 4.3. Sprint 3 Sequence Diagram

4.2.5 Completion of Sprint 3

Here are the main interfaces built during Sprint 3.

Figures 4.4 and 4.5 show the notification center and the task board with apply and completion controls.

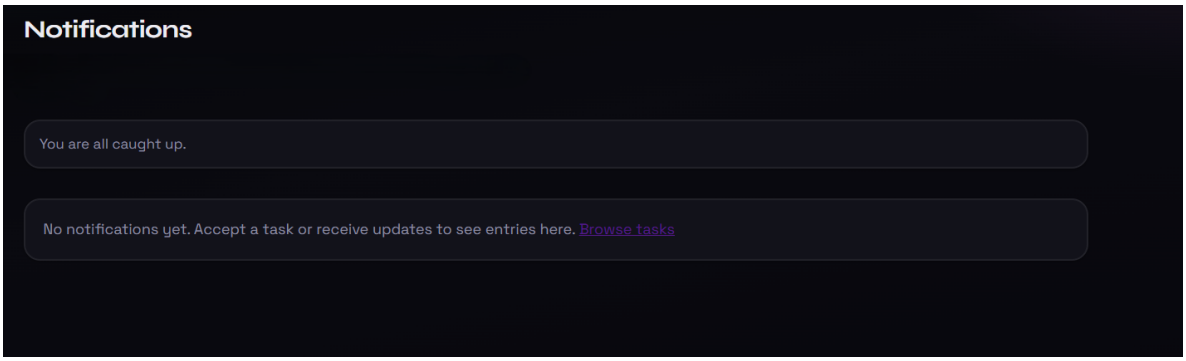


Figure 4.4. Notification Interface

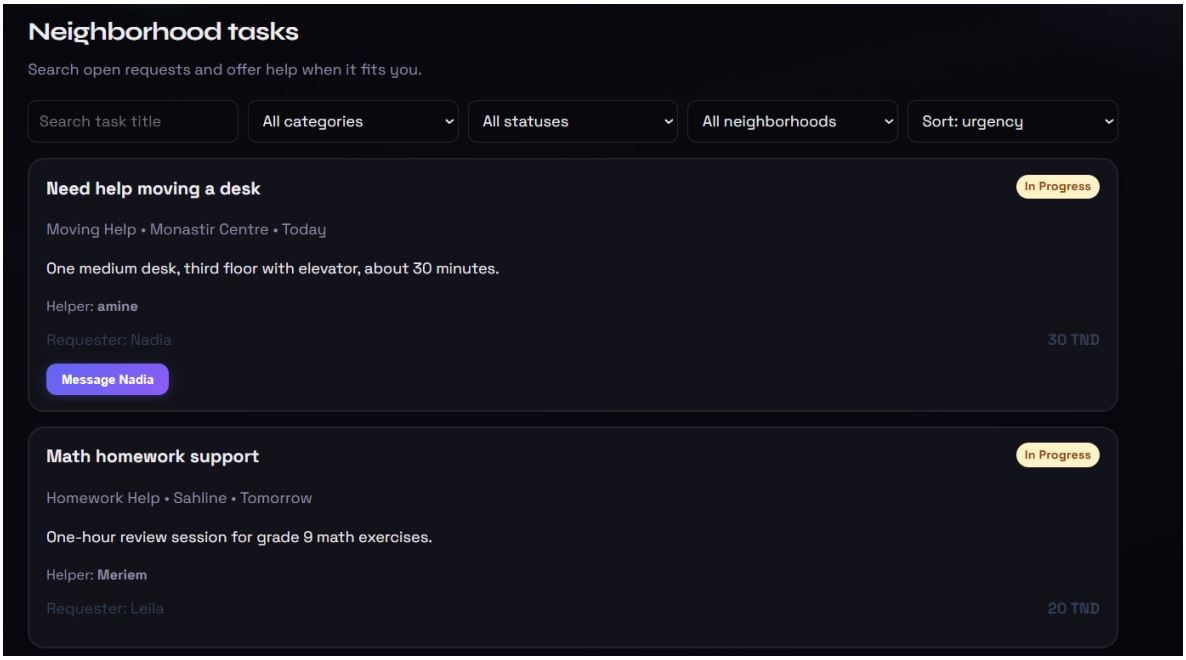


Figure 4.5. Task Board Interface

Figure 4.6 shows the atlas view with helper markers on the map.

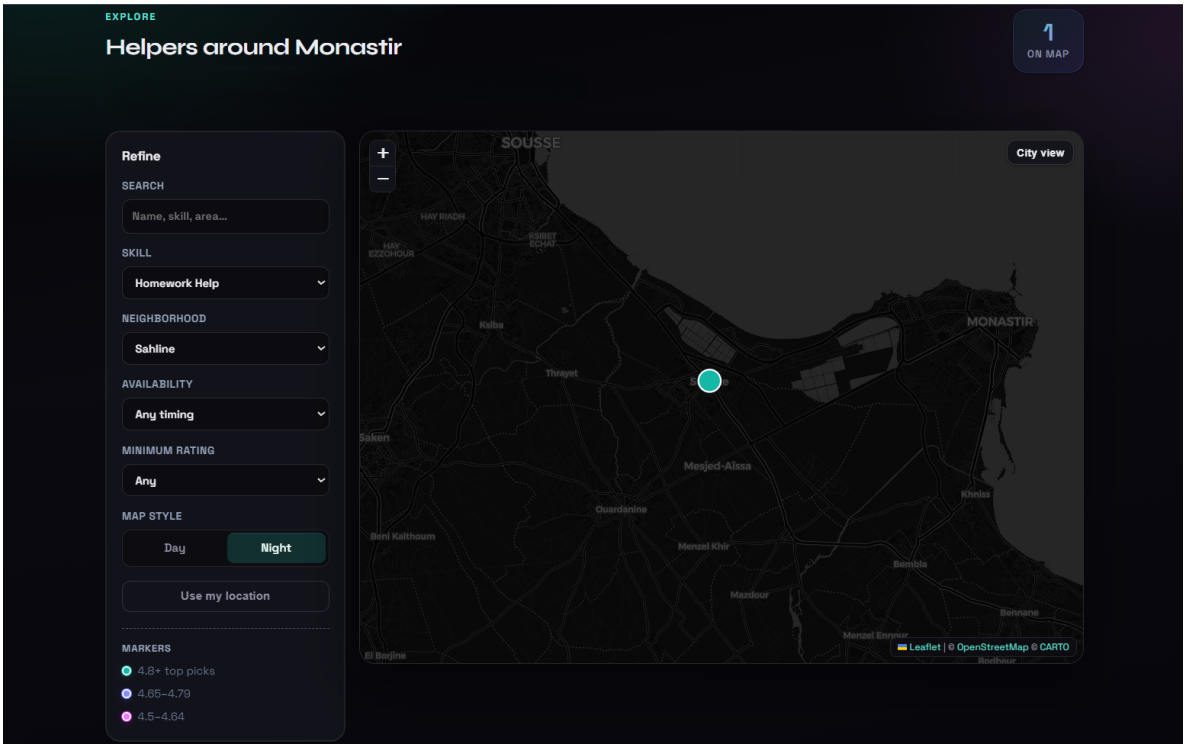


Figure 4.6. Atlas (Map View)

Figures 4.7 and 4.8 show the administrator console with the task register and accounts table.

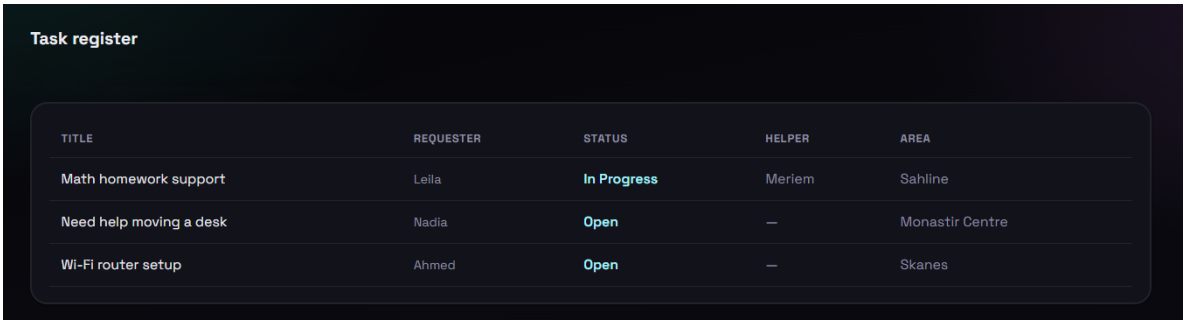


Figure 4.7. Task Register

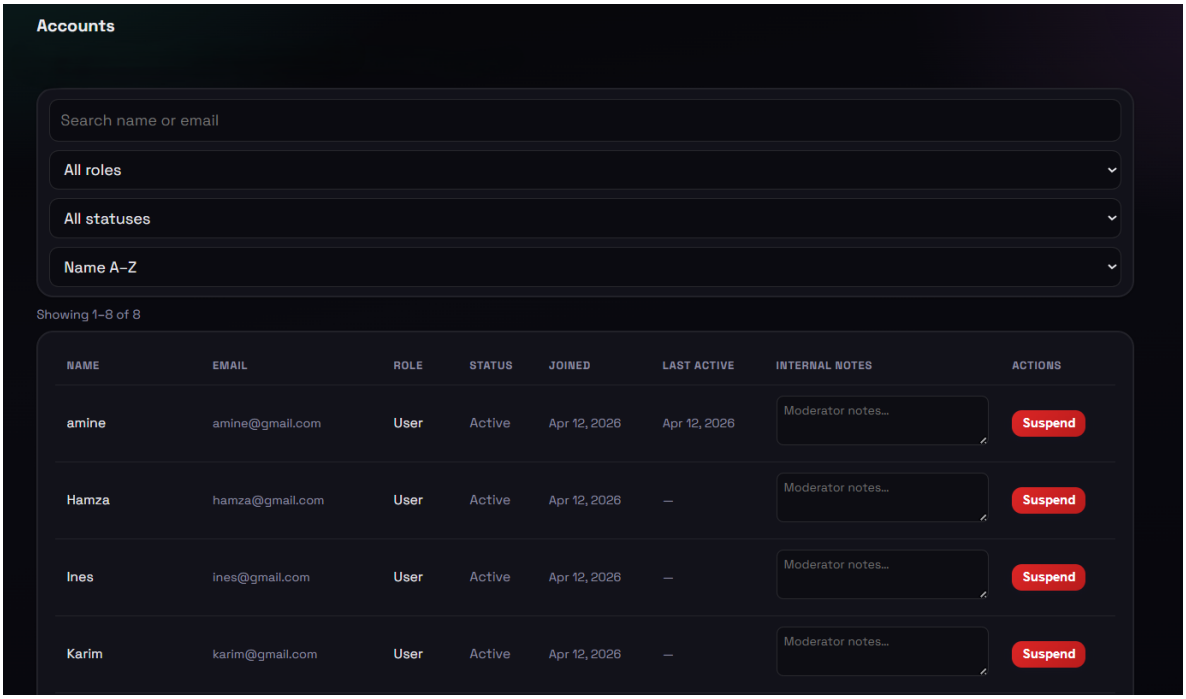


Figure 4.8. Accounts Table

4.3 Sprint 4: Rating & Review System and General Polish

4.3.1 Objective of Sprint 4

The final sprint had two goals: build the **Rate and Review** module, and give the entire application a solid round of polish. The rating system lets Help Requesters evaluate Helpers after a task is completed, giving a star score and optional written comment. This builds trust and transparency within the community. The polish phase focused on squashing bugs, improving UI consistency across all pages, and making sure everything flows cleanly for a complete and professional demo.

4.3.2 Backlog of Sprint 4

Id	Feature	User Story	Priority	Complexity
1	Rate and Review	As a Help Requester, I want to rate and review a Helper after a task is done so other users can trust the platform.	High	Medium
2	View Ratings	As a user, I want to see a Helper's ratings and reviews before accepting their help.	High	Low
3	Polish & Bug Fixes	As a user, I want the app to be consistent, smooth, and bug-free so the experience feels complete.	High	Medium

Table 4.2. Sprint 4 Product Backlog

4.3.3 Analysis of Sprint 4

Submit a Rating and Review:

1. The Help Requester opens a completed task.
2. The system prompts them to rate the Helper.
3. The requester selects a score (1 to 5 stars).
4. The requester writes an optional comment about the helper's performance.
5. The requester clicks "Submit Review".
6. If no score is selected, the system returns to step 3.
7. The system saves the rating, updates the helper's average score, and closes the prompt.

View Ratings on Profile:

1. The user navigates to a Helper's profile page.
2. The system displays the average rating and the list of reviews left by past requesters.
3. The user can scroll through individual comments and scores.

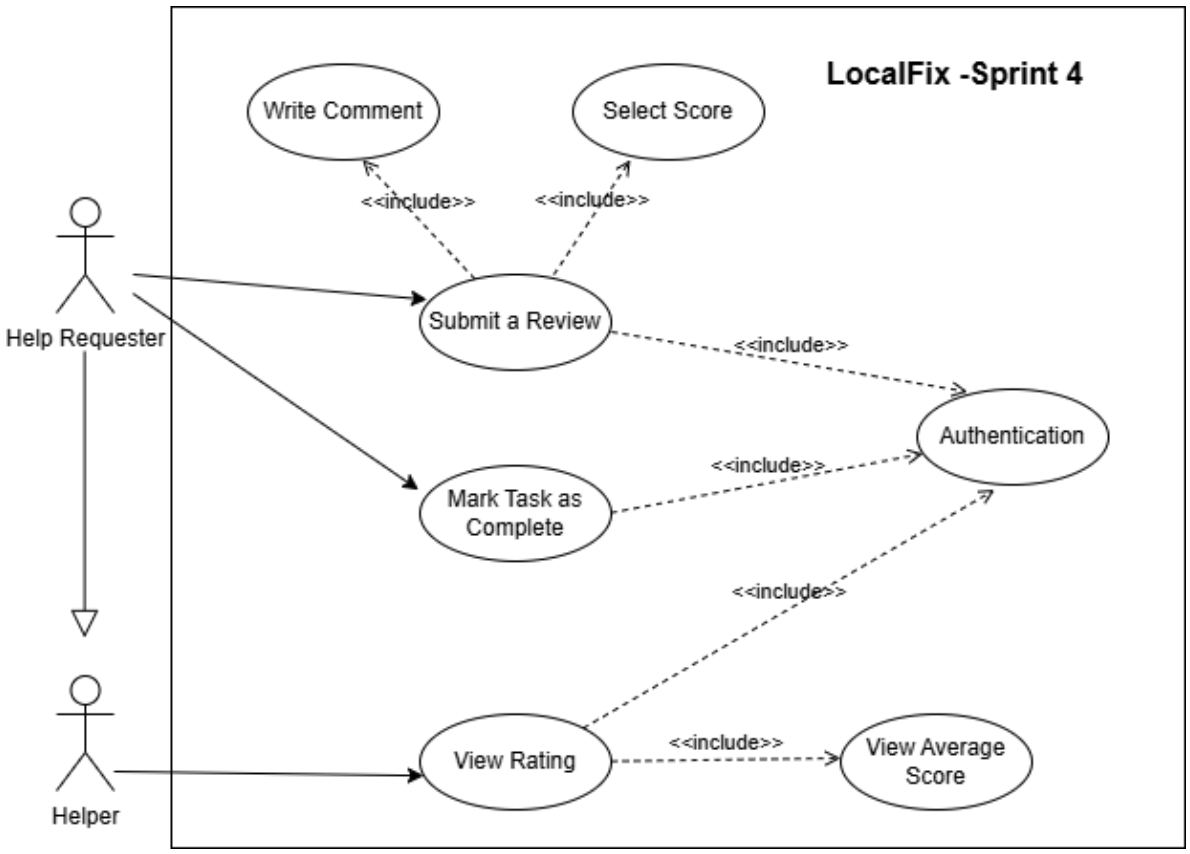


Figure 4.9. Sprint 4 Use Case Diagram

4.3.4 Design of Sprint 4

a. Class Diagram of Sprint 4

- A **Help Requester** can submit a rating and written review for a Helper once the task is marked as completed.
- A **Helper** can view their own profile, which displays an aggregated average score and all individual reviews received.
- A **Rating** is linked to exactly one completed task. It stores a numeric score (1–5 stars) and an optional comment.

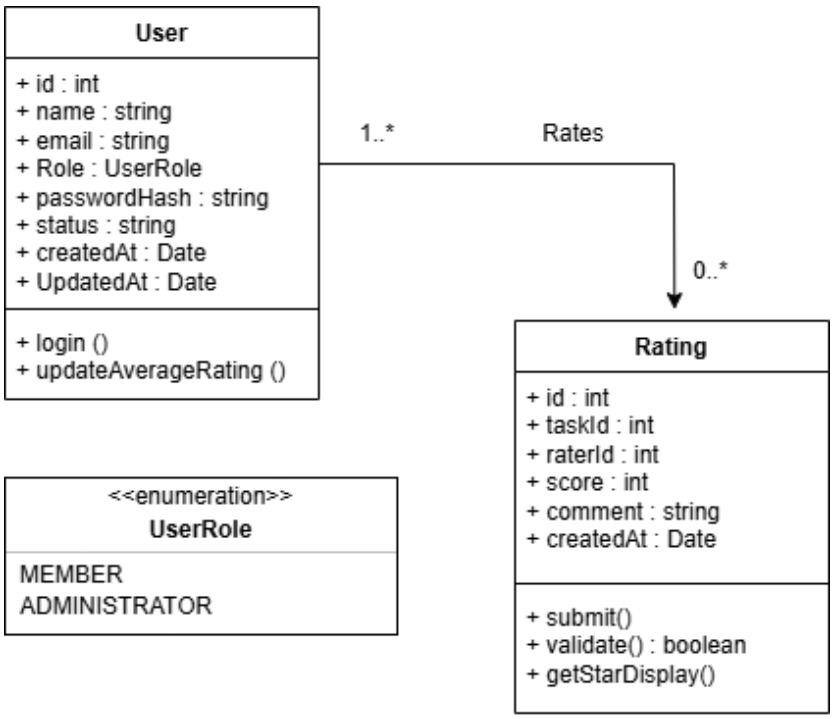


Figure 4.10. Sprint 4 Class Diagram

Figure 4.11 shows the sequence diagram for Sprint 4.

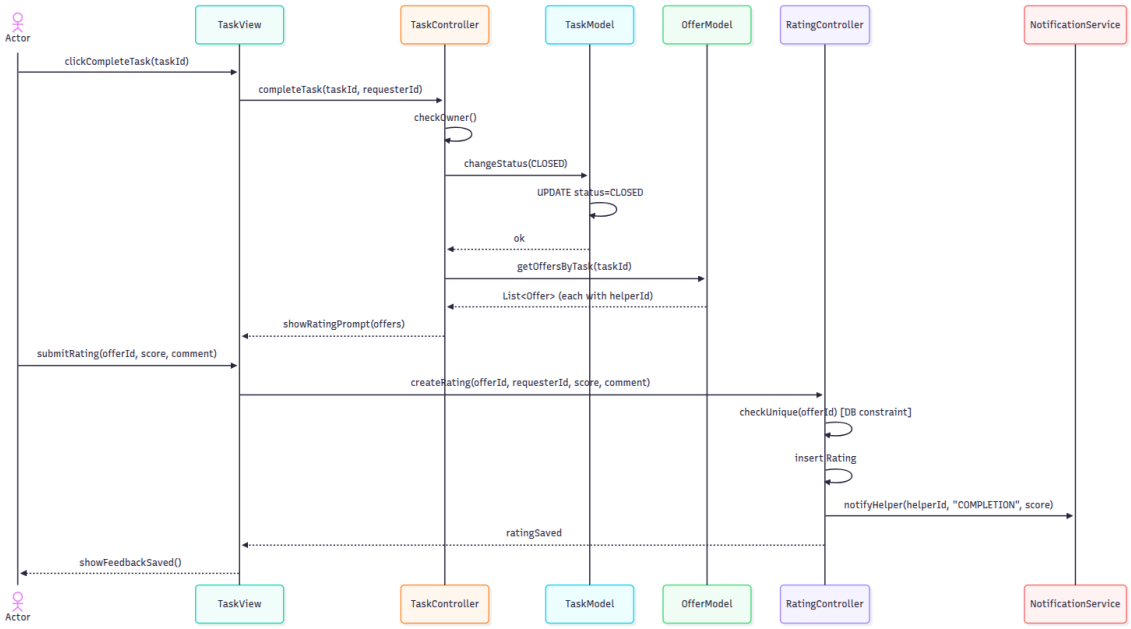


Figure 4.11. Sprint 4 Sequence Diagram

4.3.5 Completion of Sprint 4

Here are the main interfaces built during Sprint 4.

Rating Interface:

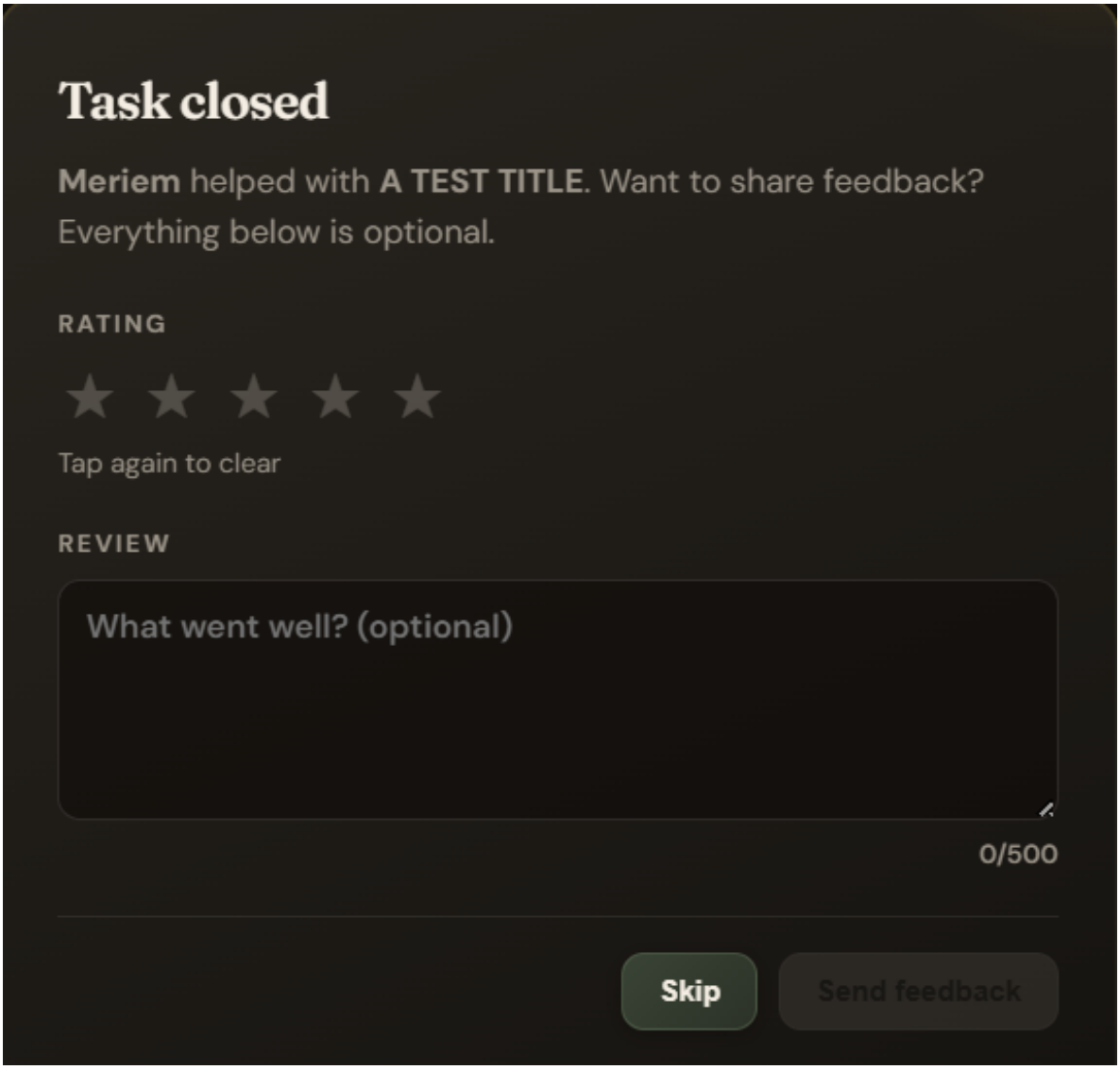


Figure 4.12. Rating Interface

Review Interface:



Figure 4.13. Review Interface

General Conclusion and Perspectives

Conclusion

This report documents the full journey of building LocalFix, from the initial idea all the way to a working application. The project was carried out at the Faculty of Sciences of Monastir (FSM) as part of our final year for the Higher Technician Diploma in Software Engineering.

Our main goal was to design and build a web-based neighborhood-help platform that genuinely makes it easier for people to connect with others in their community for small everyday tasks. To get there, we started by analyzing existing platforms like Nextdoor and PickMyTask, understanding their strengths and limitations, and using those insights to shape a cleaner, more focused solution.

We managed the project using the SCRUM framework, working in two releases and four sprints. This approach kept us organized, let us deliver working features regularly, and made it easy to adapt when things needed to change. Along the way, we covered analysis and requirements specification, architectural design, full-stack implementation, and a final polish phase.

What we ended up building is a platform with a clean task board for posting and browsing help requests, an integrated messaging system for communication, a reporting and moderation system for platform safety, an admin dashboard with live statistics and account management, a map-based discovery view for finding nearby helpers, and a rating and review system to build trust within the community.

This project was genuinely a great learning experience. We got to apply what we've been studying to a real problem, and we came out of it with stronger skills in software design, full-stack web development, project management, and working as a team.

Perspectives

Short-Term Perspectives

- **In-app verification tools:** Adding features like photo uploads for task progress, QR code completion confirmation, or GPS check-ins would help verify that work was actually done and reduce disputes.
- **Mobile application:** A native iOS and Android app would make LocalFix far more accessible, enabling real-time location-based task discovery and instant messaging on the go.

Long-Term Perspectives

- **Payments integration:** Integrating a payment solution like Stripe or Paystack with escrow support would enable real monetization for paid tasks, with the admin releasing funds on completion.
- **AI-powered matching:** Machine learning algorithms could power smarter task recommendations based on skills, location, and past success, as well as dynamic pricing and demand forecasting.

Online Resources

The following online resources were consulted during the development of this project.

1. **Nextdoor**
   https://about.nextdoor.com/

2. **PickMyTask**
   https://pickmytask.com/about

3. **React**
   https://react.dev/

4. **Visual Studio Code**
   https://code.visualstudio.com/

5. **MySQL**
   https://dev.mysql.com/doc/

6. **Node.js**
   https://nodejs.org/en/docs/

7. **MongoDB**
   https://www.mongodb.com/docs/

8. **TeXstudio**
   https://www.texstudio.org/

9. **Draw.io**
   https://www.drawio.com/about

10. **Jira**
    https://www.atlassian.com/software/jira

11. **Microsoft Teams**
    https://teams.live.com/free

12. **SCRUM**
    https://interqualitybg.com/images/SCRUM_METHODOLOGY_InterQuality_image_2.jpg

13. **MVC Architecture**
    https://media.geeksforgeeks.org/wp-content/uploads/20240704102850/MVC-Architecture.webp

14. **Event-Driven Architecture**
    https://miro.medium.com/v2/

15. **3-Tier Architecture**
    https://positiwise.com/blog/wp-content/uploads/2023/08/3-tier-architecture.jpg